Lab 3: Build & Test Pipeline (CI)

I. Learning Objectives

After completing this lab, you will be able to:

- Create a GitHub Actions CI workflow from scratch.
- Configure the workflow to run automatically on Pull Requests.
- Implement steps to install dependencies (`npm install`) and run tests (`npm test`).
- Implement steps to build a Docker image.
- Securely push your built Docker image to Docker Hub using GitHub Secrets.

II. Prerequisites

- **Lab 2 Completed:** Your project must have a working `Dockerfile` and `docker-compose.yml` in the `main` branch.
- **Docker Hub Account:** You must have a Docker Hub account. You will need your Username and a **Personal Access Token (PAT)**.

> **Important:** Do not use your password. Go to Docker Hub > Account Settings > Security > New Access Token. Create a token and save it.

- **New Branch:** All work for this lab MUST be done on a new feature branch.

```
# Make sure you're on main and have the latest changes
$ git checkout main $ git pull origin main



# Create a new branch for this lab's work
$ git checkout -b feature/add-ci-pipeline
```

III. Part A: Create the Basic CI Workflow

Our first goal is to create a workflow that automatically runs our tests (`npm test`) every time someone opens a Pull Request.

Step A.1: Create the Workflow File

GitHub Actions workflows are defined in YAML files, which must be in a specific folder.

1. In the **root** of your project, create a new folder named `.github`.
2. Inside `.github`, create another folder named `workflows`.
3. Inside `.github/workflows`, create a new file named `ci.yml`.

Your project structure should look like this:

```
/
├── .github/
│   └── workflows/
│       └── ci.yml   <-- YOU ARE CREATING THIS
├── src/
├── Dockerfile
├── docker-compose.yml
└── package.json
```

Step A.2: Define the Name and Trigger

Open `ci.yml` and add the following content. This defines the name of your workflow and *when* it should run.

```yaml
name: CI Pipeline


# This workflow runs on two triggers:
# 1. When code is pushed to the 'main' branch
```

```yaml
# 2. When a Pull Request is opened/updated against the 'main' branch
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
```

> The `pull_request` trigger is our **gatekeeper**. It will run our tests **before** we merge, preventing broken code from entering `main`.

Step A.3: Define the `build-and-test` Job

Next, we define our `jobs`. We will start with one job called `build-and-test`. This job needs a database to run the tests, so we will use `services` to start a Postgres container.

```yaml
# ... (name: and on: are above) ...

jobs:
  build-and-test:
    name: Build & Test
    runs-on: ubuntu-latest

    # This starts a Postgres database *for this job*
    services:
      db:
        image: postgres:14-alpine
        environment:
          - POSTGRES_USER=myuser
          - POSTGRES_PASSWORD=mypass
```

```
          - POSTGRES_DB=mydatabase
        ports:
          - 5432:5432
        # This health check ensures the DB is ready before tests
run
        options: --health-cmd pg_isready --health-
interval 10s --health-timeout 5s --health-retries 5

    steps:
      # Step 1: Get our project code onto the runner
        - name: Checkout code
          uses: actions/checkout@v3

      # Step 2: Install Node.js v18
        - name: Setup Node.js
          uses: actions/setup-node@v3
          with:
            node-version: '18'
            cache: 'npm' # Caches npm packages for faster
builds

      # Step 3: Install our app's dependencies
        - name: Install dependencies
          run: npm install

      # Step 4: Run the tests. We must provide the DB env vars!
        - name: Run tests
          run: npm test
          env:
```

```
                DB_HOST: localhost # GitHub maps service
containers to localhost
                DB_USER: myuser
                DB_PASSWORD: mypass
                DB_NAME: mydatabase
```

IV. Part B: Test the CI Pipeline

Let's see our new pipeline in action.

Step B.1: Commit and Push

Save your `ci.yml` file. Now, commit and push it on your branch.

```
# 1. Add your new workflow file
$ git add .github/workflows/ci.yml

# 2. Commit it
$ git commit -m "ci: Add initial build and test
workflow"

# 3. Push your branch
$ git push origin feature/add-ci-pipeline
```

Step B.2: Open a Pull Request

1. Go to your group's GitHub repository.
2. You will see the prompt to open a Pull Request for `feature/add-ci-pipeline`. Click it.
3. Add your teammates as reviewers and click "**Create Pull Request**".

Step B.3: Observe the "Action"

As soon as you create the PR, the `pull_request` trigger fires.

1. On your PR page, you will see a new "check" appear at the bottom, starting with a yellow spinning icon.
2. Click the "Actions" tab of your repository to see the "CI Pipeline" workflow running.
3. Click on it, then click the "**Build & Test**" job to see the live logs.
4. You will see it start the `db` service, check out your code, install Node, run `npm install`, and finally `npm test`.

> **Task:** If your job fails (a Red X), click on the failing step (e.g., "Run tests") and read the logs. Debug the problem (it's often a missing `env` variable), fix it, commit, and push again. The PR will automatically re-run the check.

Your goal is to get a **Green Checkmark** . Do not merge the PR yet!

V. Part C: Build and Push Docker Image

Now we will add the final steps to our job: building the Docker image and pushing it to Docker Hub. This ensures our `Dockerfile` is always valid.

Step C.1: Add GitHub Secrets for Docker Hub

We must never write our passwords in the `ci.yml` file. We use GitHub Secrets.

1. In your group's GitHub repo, go to `Settings` > `Secrets and variables` > `Actions`.
2. Click `New repository secret`.
3. **Secret 1:**
    - Name: `DOCKER_HUB_USERNAME`
    - Value: `(Your Docker Hub username)`

4. **Secret 2:**
    - Name: `DOCKER_HUB_PASSWORD`
    - Value: `(Your Docker Hub Personal Access Token you created)`

Step C.2: Add Docker Steps to `ci.yml`

Go back to your `ci.yml` file on your `feature/add-ci-pipeline` branch. Add the following steps **after** the `npm test` step.

```
# ... (steps: ... npm test ... are above) ...

    # Step 5: Log in to Docker Hub
   - name: Log in to Docker Hub
     uses: docker/login-action@v2
     with:
       username: ${{ secrets.DOCKER_HUB_USERNAME }}
       password: ${{ secrets.DOCKER_HUB_PASSWORD }}

    # Step 6: Build and push the Docker image
   - name: Build and push Docker image
     uses: docker/build-push-action@v4
     with:
        context: . # Build from the Dockerfile in the root
        push: true # Actually push the image
        tags: ${{ secrets.DOCKER_HUB_USERNAME
}}/devops-group-1:latest
```

> **Critical:** Change `devops-group-1` to your group's repository name! The tag format should be `your-dockerhub-username/your-repo-name:latest`.

Step C.3: Commit and Push the Final Changes

You have updated your `ci.yml`. Let's commit and push this final change.

```
# 1. Add the updated file
$ git add .github/workflows/ci.yml

# 2. Commit it
$ git commit -m "ci: Add Docker build and push steps"

# 3. Push your branch
$ git push origin feature/add-ci-pipeline
```

Step C.4: Verify the Full Pipeline

Go back to your Pull Request. The workflow will run again automatically.

Task: Watch the workflow logs. This time, you should see all steps pass:

1. Checkout
2. Setup Node
3. Install dependencies
4. Run tests
5. **Log in to Docker Hub** (should pass)
6. **Build and push Docker image** (should pass)

After it finishes, go to your Docker Hub account and verify that the new image (e.g., `your-username/devops-group-1:latest`) is there!

Step C.5: Merge the PR

Your pipeline is now complete. It has proven that your code can be tested and built into a deployable artifact (the Docker image).

Have your teammates review and **Approve** the PR. The Team Leader can then **Merge** it.